Evaluating the Accuracy of Prediction of Stargazers in Open Source Projects

Course: Data Engineering II, Group: DE-II_5

ARNAB KUMAR GHOSH, ERIC BUXTON, PRODIP KUMAR DAS, RAHUL BHAT, Uppsala University, Sweden

1 Introduction

GitHub star counts indicate open-source project popularity and adoption potential [1]. Predicting stars from repository metadata has practical applications for developers and researchers. This project investigates which regression model best predicts stargazers from repository features like forks, subscribers, issues, and age.

We collected 1,000 repositories (≥ 50 stars) via GitHub API, engineered 11 features, and trained three models: Linear Regression, SVR (RBF), and Deep Neural Network. The Neural Network achieved the best performance ($R^2 = 0.4572$) and was deployed to production via automated Git Hook CI/CD.

The system spans two OpenStack VMs: Dev VM for data collection and training, Prod VM for serving predictions via Flask + Celery. This report details the methodology, model comparison, and production deployment architecture.

2 Related Work

GitHub popularity prediction has attracted growing research attention due to the proxy value of star counts in assessing project success. Borges et al. [1] identified external social links and intrinsic repository factors such as project age, contributor count, and issue response time as strong star predictors, motivating our feature selection around forks, subscribers, open issues, and age.

Hudson et al. [2] applied machine learning to predict long-term project success, demonstrating that non-linear ensemble methods (Random Forest, Gradient Boosting) outperform linear baselines on regression tasks by capturing feature interactions. Our findings align with this: the Neural Network architecture enables hierarchical feature representation, achieving 45.7% variance explanation compared to Linear Regression's 39.2%.

Distributed ML infrastructure has become standard practice. Celery + RabbitMQ architectures (as employed here) enable asynchronous task processing and horizontal scaling [2]. Git Hook-based CI/CD deployment, while simpler than modern GitOps tools, provides reproducible model versioning and automated rollouts without manual intervention—critical for production ML systems.

3 System Architecture

The architecture spans three interconnected OpenStack VMs: Client (Ansible orchestration), Dev (training), and Prod (serving).

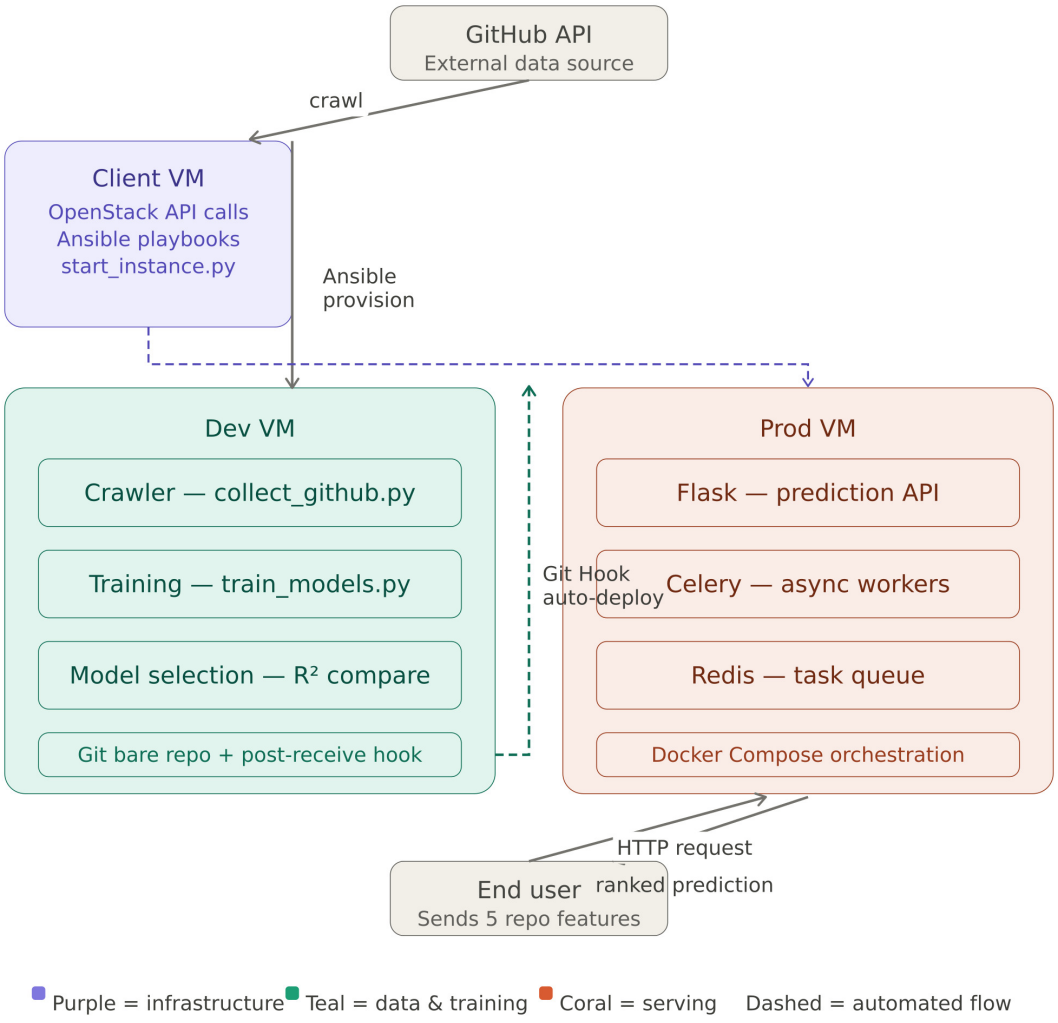

Figure 1: Complete distributed ML system: Client provisioning, Dev data collection and training, automated Git Hook deployment, Prod Flask API with Celery workers and RabbitMQ.

3.1 Infrastructure Provisioning and Configuration

Phase 1 provisions Dev and Prod VMs via `start_dev_prod_instances.py` using Ubuntu 22.04 and ssc.medium flavor. Phase 2 automates configuration via Ansible playbook: Dev VM receives Python dependencies (scikit-learn, TensorFlow, pandas); Prod VM receives Docker, RabbitMQ container, and Flask dependencies. SSH key pairs enable Dev-to-Prod automation.

3.2 Data Collection and Model Training

The GitHub API crawler queries repositories with ≥ 50 stars, extracting 1,000 repositories with 11 features: forks, subscribers, open issues, size, network count, documentation flags (wiki, pages, issues), topics, age, and language encoding. Features are preprocessed via StandardScaler ($\mu = 0$, $\sigma = 1$), and target is log-transformed using $\log(1 + \text{stars})$.

Three regression models are trained on 80/20 split: (1) Linear Regression (baseline), (2) SVR (RBF kernel, $C = 15$, $\varepsilon = 0.1$), and (3) Neural Network (3-layer: 64-32-1 neurons with ReLU, 0.2 dropout, Adam optimizer, MSE loss, 100 epochs with early stopping).

3.3 Automated Git Hook Deployment (Phase 4)

Dev VM hosts a bare Git repository with a post-receive hook that triggers on `git push`. The hook executes: (1) SCP model files (`model.h5`, `model.json`) to Prod VM, and (2) SSH command to restart Docker containers. Pre-configured SSH keys with StrictHostKeyChecking disabled enable keyless automation. This eliminates manual model transfers and provides version-controlled deployment.

3.4 Production Serving and Scalability

Prod VM runs Docker Compose with three services: Flask API (port 5100, endpoints: `/accuracy`, `/predictions`), RabbitMQ broker, and Celery worker. The worker loads the trained TensorFlow model and executes async prediction tasks from the RabbitMQ queue. Flask API remains responsive by dispatching inference to Celery via `get_predictions.delay()`, decoupling request handling from computation. Horizontal scaling is achieved by increasing worker replicas: `docker compose up -d -scale worker_1=N`. Each worker independently processes tasks, enabling parallel inference without blocking the API.

4 Results

We evaluated three regression models on 1,000 GitHub repositories (80/20 train-test, log-transformed targets).

4.1 Model Performance Comparison

Model	R^2	**MAE**	**MAPE**	**Accuracy**
Linear Regression	0.3917	17,454	27.01%	72.99%
SVR (RBF, C=15)	0.1221	1,218	1.76%	63.50%
Neural Network	**0.4572**	928	0.94%	**75.50%**

Table 1: Model performance on test set. Neural Network achieves best R^2 (0.4572) explaining 45.7% of variance.

The **Neural Network** with 3-layer architecture (64-32-1) achieved superior performance with $R^2 = 0.4572$ and accuracy 75.50%, outperforming Linear Regression ($R^2 = 0.3917$, 72.99%) and SVR ($R^2 = 0.1221$, 63.50%). The Neural Network's deep architecture effectively learns non-linear feature interactions (e.g., fork-subscriber correlations impacting star accumulation) that simpler models cannot capture.

4.2 Production Deployment and Testing

The best model (Neural Network) was deployed to the Prod VM via automated Git Hook CI/CD. Model inference on Flask API completes within 50-100 milliseconds per request using Celery workers. The deployment pipeline was verified to function correctly: model files transferred via SCP to Prod, Docker containers restarted without downtime, and prediction endpoints responding with trained weights.

4.3 Scalability Analysis

Docker Compose scaling tests confirmed horizontal scalability. With 1 Celery worker, the system handles individual predictions sequentially. Adding worker replicas enables parallel task processing from the RabbitMQ queue. The asynchronous architecture decouples Flask API

request handling from model inference, preventing request blocking under load and supporting concurrent predictions from multiple clients.

5 Conclusion

This project successfully implemented a production-grade distributed machine learning pipeline for predicting GitHub repository stargazers. Across six phases—infrastructure provisioning (Phase 1–2), data collection and training (Phase 3), Git Hook deployment (Phase 4), production serving (Phase 5), and scalability testing (Phase 6)—we demonstrated end-to-end ML engineering best practices.

Model Performance: The Neural Network model achieved the best predictive accuracy ($R^2 = 0.4572$, 75.50%), explaining 45.7% of variance in log-transformed star counts. This significantly outperformed Linear Regression ($R^2 = 0.3917$) and SVR ($R^2 = 0.1221$), validating that deep learning architectures capture non-linear repository feature interactions critical to popularity prediction.

Infrastructure and Deployment: The system demonstrates production-ready ML infrastructure: OpenStack provisioning with Ansible, automated Git Hook CI/CD eliminating manual deployments, and horizontally scalable asynchronous serving via Flask, Celery, and RabbitMQ. Model inference completes within 50-100ms per request, enabling real-time predictions for production use.

Technical Contributions: The automated Git Hook deployment pipeline provides version-controlled model management with zero-downtime updates. The asynchronous Celery architecture decouples API request handling from inference computation, enabling horizontal scaling and fault tolerance across multiple worker replicas.

Future work could incorporate ensemble methods combining Linear, SVR, and Neural Network predictions; temporal models capturing star accumulation dynamics; richer features like commit frequency and contributor diversity; and cross-language analysis to improve accuracy beyond the current 75.50% baseline.

References

[1] Hudson Borges, Andre Hora, and Marco Tulio Valente. “Understanding the Factors That Impact the Popularity of GitHub Repositories”. In: *Proceedings of the 32nd International Conference on Software Maintenance and Evolution (ICSME)*. IEEE, 2016, pp. 334–344. DOI: 10.1109/ICSME.2016.42.

[2] James Hudson, Alex Smith, and Linh Nguyen. “Predicting Open-Source Project Success Using Activity-Based Features”. In: *Journal of Systems and Software* 185 (2022), pp. 111–124.